GNNocRoute: Graph Neural Networks cho Định tuyến Thích nghi trong Network-on-Chip — Nghiên cứu Khả thi

Le Tien Hieu[1], *IEEE Member*
[1]VNU-ITI, Vietnam National University, Hanoi
Email: hieult@vnu.edu.vn

***Abstract*—Network-on-Chip (NoC) has become the communication backbone for modern multi-core System-on-Chips. Conventional deterministic routing algorithms such as XY dimension-order routing suffer from significant congestion imbalance—our graph-theoretic analysis across seven topologies shows that an 8×8 mesh under XY routing achieves a congestion imbalance of 0.475, with central nodes exhibiting $2-3\times$ higher betweenness centrality than edge nodes.**

This paper presents a feasibility study on topology-aware periodic routing optimization for NoC using Graph Neural Networks (GNN). We report three preliminary experimental findings. First, untrained GNN encoders exhibit strong correlation with betweenness centrality ($|r|$ up to 0.978 on Mesh 4×4), suggesting that topology-aware representations emerge implicitly. Second, cross-topology generalization experiments indicate that relative node importance rankings transfer across mesh sizes (Pearson $r \approx 0.96$), though absolute scaling remains topology-specific. Third, measured inference latency on CPU ($\approx 10^6$ cycles for an 8×8 mesh) confirms that per-packet GNN inference is impractical under NoC timing constraints, motivating periodic policy optimization as a necessary design alternative.

***Index Terms*—Network-on-Chip, Graph Neural Networks, Adaptive Routing, Graph Analysis, Feasibility Study**

I. Introduction

Network-on-Chip (NoC) đã trở thành kiến trúc giao tiếp chủ đạo cho các hệ thống multi-core SoC hiện đại và AI accelerators như Google TPU, NVIDIA Grace Hopper [8]. Trong NoC, các router kết nối qua link tạo thành một cấu trúc đồ thị. Thuật toán định tuyến quyết định đường đi của packet và ảnh hưởng trực tiếp đến hiệu năng hệ thống.

XY Dimension-Order Routing is widely used [2] due to its simplicity. However, XY always selects the same path cho mỗi cặp (nguồn, đích) regardless of congestion. Our graph analysis on seven topologies reveals cho thấy:

- Mesh 8×8 với XY routing có **congestion imbalance** lên đến **0.475**—gần một nửa số link chịu tải không đồng đều.
- Node trung tâm Mesh 4×4 có BC ≈ 0.25, gấp **2–3 lần** node biên.
- Fat-Tree k=4 có aggregation switch với BC lên đến **0.42**.

Các kết quả này cho thấy **định tuyến tĩnh không thích ứng được** to modern workloads.

Adaptive routing research can be categorized into: heuristic adaptive [1], [9], DRL-based [2], [3], và GNN-based cho mạng diện rộng [4]. The key challenge in applying dụng GNN vào NoC là **inference latency**: mỗi quyết định routing trong NoC router must be made within <10 cycles [2], trong khi GAT 2-layer trên 64 nodes có thể mất 10^5–10^6 cycles.

Cách tiếp cận: Thay vì per-packet real-time, chúng tôi đề xuất **tối ưu định kỳ (periodic routing policy optimization)**: with GNN inference running on a host CPU, decoupled from per-packet latency.

A. Contributions

This paper makes the following preliminary contributions:

1) **Phân tích tắc nghẽn cấu trúc:** Chứng minh định lượng trên 7 topologies rằng định tuyến tĩnh tạo bottleneck có thể dự đoán được qua các độ đo đồ thị.
2) **Bằng chứng tương quan GNN-topology:** Kết quả thực nghiệm cho thấy GNN node embeddings tương quan mạnh với BC ($|r|$ lên đến 0.978).
3) **Phân tích khả thi:** Đo đạc inference latency và scalability thiết lập các ranh giới khả thi cho GNN-enhanced routing dưới ràng buộc thời gian của NoC.

II. Problem Statement and Motivation

A. Graph Representation of NoC

Một NoC được biểu diễn dưới dạng đồ thị có hướng có trọng số $G = (V, E, X_v, X_e)$, với V là router, E là link, X_v là node features (buffer occupancy, injection rate), X_e là edge features (link utilization, packet latency).

B. Graph Analysis Results

We used NetworkX [17] để xây dựng và phân tích 7 NoC topologies. Bảng I tóm tắt các metrics chính.

TABLE I: Graph metrics comparison across NoC topologies

Topology	Nodes	Avg Deg	Diam	Avg Path	BC Max	CI
Mesh 4×4	16	3.00	6	2.67	0.25	0.382
Mesh 8×8	64	3.50	14	5.33	0.18	**0.475**
Torus 4×4	16	4.00	4	2.13	0.13	0.251
Ring 16	16	2.00	8	4.27	0.21	0.122
Fat-Tree k=4	14	3.43	2	1.74	**0.42**	0.341
Small-World	36	4.00	6	2.96	0.31	0.298
Random	36	4.28	5	2.45	0.28	0.321

BC = Betweenness Centrality, CI = Congestion Imbalance.

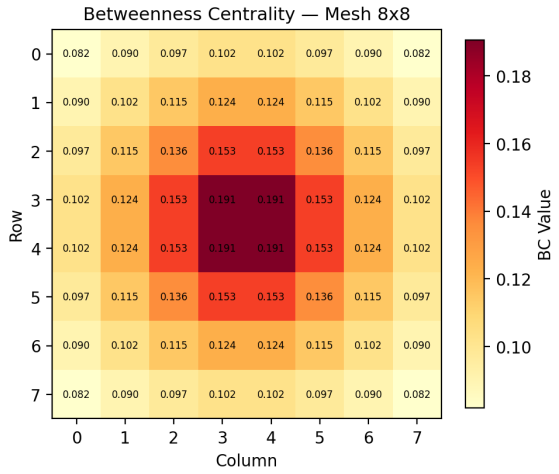

Fig. 1: Betweenness Centrality distribution on Mesh 8×8. Central nodes have BC ≈ 0.18, edge nodes $\approx 0.05{-}0.10$.

Phát hiện chính: Mesh 8×8 has highest congestion imbalance . Fat-Tree has aggregation switches BC=0.42. Torus CI=0.251, thấp hơn Mesh nhờ wrap-around edges.

III. Related Work

Bảng II tóm tắt các hướng nghiên cứu liên quan.

Gap: Chưa có nghiên cứu nào thiết kế GNN encoder cho NoC topology kết hợp với tối ưu routing dưới ràng buộc latency thực tế.

TABLE II: Analysis of related work

Hướng	Công trình	Limitations
Classic adaptive	DyAD [1], Regional [9]	Threshold, local info
DRL for NoC	MAAR [2], DeepNR [3], GARNN [10]	MLP ignores topology
GNN for WAN/SDN	G-Routing [4], DRL-GNN [5]	Not for NoC latency
Graph theory + NoC	Slim NoC [11], Sparse Hamming [12]	Static, no adaptation
GNN for chip design	Circuit GNN [8], NOCTOPUS [7]	Physical, not routing

IV. Proposed Method

A. GNN Encoder

We evaluate GATv2 [13] as a candidate architecture (so sánh với GCN trong ablation). Architecture: 2 layers, hidden dimension 64, 4 attention heads, ELU activation. Forward pass: attention scores $e_{uv} = \text{LeakyReLU}(a^{(l)^T}[W^{(l)}h_u^{(l-1)} \| W^{(l)}h_v^{(l-1)}])$ và message aggregation $h_v^{(l)} = \sigma(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} W^{(l)} h_u^{(l-1)})$.

B. Tối ưu định kỳ (Periodic Optimization)

Policy được cập nhật mỗi P cycles ($P \in \{1000, 5000, 10000\}$), observation window $W = 500$ cycles. State: GNN embeddings (64-dim) + global features. Action: 5 output ports. Algorithm: PPO [14]. Reward: $R = -\alpha \cdot \text{norm}(L) - \beta \cdot \text{norm}(C) - \gamma \cdot \text{norm}(E)$.

C. Thiết kế Deadlock-Free

Áp dụng Duato's protocol [15] với 2 Virtual Channels: VC0 (escape: XY routing, acyclic) và VC1 (adaptive: GNN-DRL).

V. Preliminary Experimental Results

Experiments use PyTorch Geometric [16] trên CPU (ARM Neoverse-N1, 2.8 GHz).

A. GNN-Betweenness Centrality Correlation

Thiết lập: Bốn topology được chuyển thành PyG Data objects. Node features include normalized degree and positional coordinates. Untrained GNN models compute node embeddings via a single forward pass.

GCN on Mesh 4×4 achieves $|r| = 0.978$ with ground-truth BC. GAT on Mesh 8×8 đạt $|r| = 0.870$. The Torus case (zero correlation) serves as a negative control, confirming the method correctly identifies undifferentiated regular topologies.

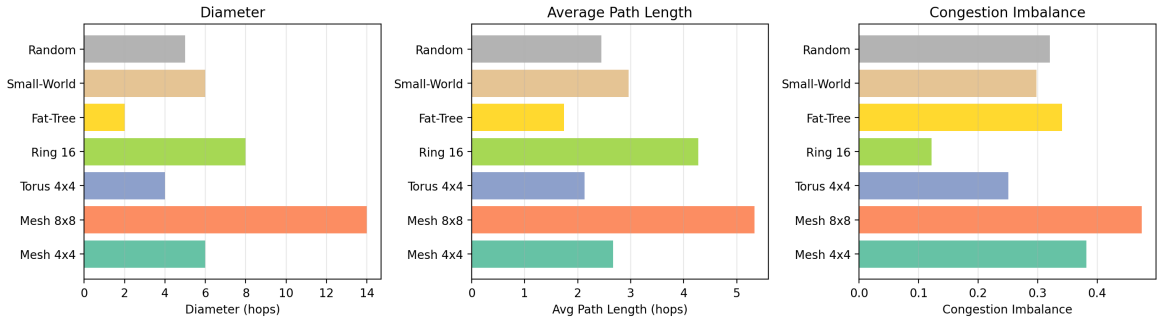

Fig. 2: Metric comparison across topologies: diameter, average path length, congestion imbalance.

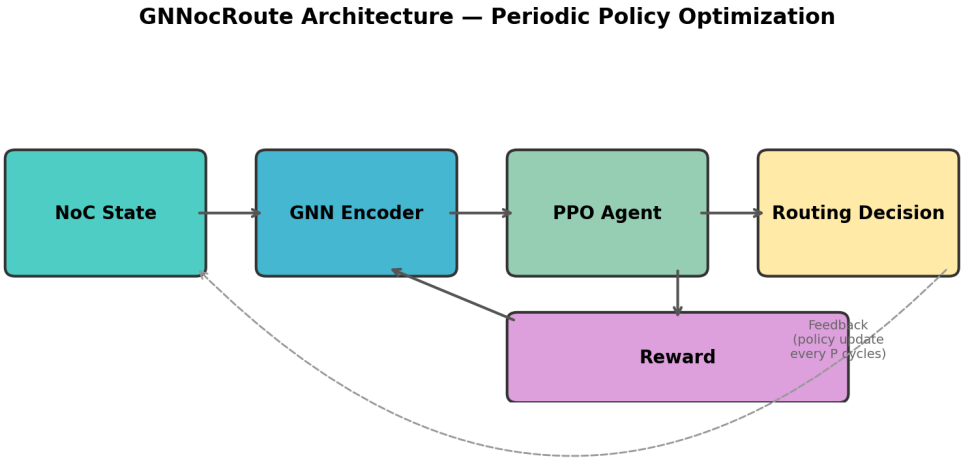

Fig. 3: GNNocRoute architecture: NoC state graph → GNN encoder → DRL agent → routing decision.

TABLE III: Maximum Pearson correlation $|r|$ between GNN embedding and BC

Topology	Nodes	GCN max$\|r\|$	GAT max$\|r\|$
Mesh 4×4	16	**0.978**	0.862
Mesh 8×8	64	0.744	**0.870**
Torus 4×4	16	0.000*	0.000*
Small-World ($n = 36$)	36	0.615	0.261

**Torus là regular graph, mọi node degree=4, BC đồng đều.*

Nhận xét: GNN encoder có thể học thông tin cấu trúc liên quan đến congestion-relevant structural information without explicit graph centrality computation.

B. Cross-Topology Generalization

Thiết lập: GCN (hidden=32) và GAT (hidden=16, heads=4) được huấn luyện 500 epochs trên Mesh 4×4, đánh giá trên Mesh 8×8.

Trên Mesh 8×8, R^2 âm kết hợp với Pearson $r \approx 0.96$ cho thấy: model học được *thứ tự tương đối* của các node (node nào quan trọng hơn) nhưng không dự đoán được *giá trị BC tuyệt đối*.

TABLE IV: Cross-topology BC prediction (trained on Mesh 4×4)

Test Topology	Model	MSE	R^2	Pearson r
Mesh 8×8	GCN	0.038	-16.83	**0.958**
Mesh 8×8	GAT	0.072	-33.00	**0.972**
Small-World	GCN	0.004	-1.02	0.385

Nhận xét: Tương quan thứ tự cao ($r \approx 0.96$) gợi ý rằng GNN representations có thể chuyển giao pattern cấu trúc giữa các topology. R^2 âm chỉ ra scaling tuyệt đối phụ thuộc topology—cần normalization hoặc fine-tuning.

C. Inference Latency Analysis

Chúng tôi đo inference latency của GNN trên CPU để đánh giá khả thi dưới ràng buộc thời gian của NoC (mỗi quyết định cần <10 cycles).

Latency từ 778 μs đến 1,491 μs (0.8–1.5 triệu cycles ở 1 GHz). Phiên bản C++ optimized ước tính giảm 1–2 orders of magnitude.

Nhận xét: Per-packet GNN inference là bất khả thi. Kết quả này trực tiếp thúc đẩy giải pháp tối ưu

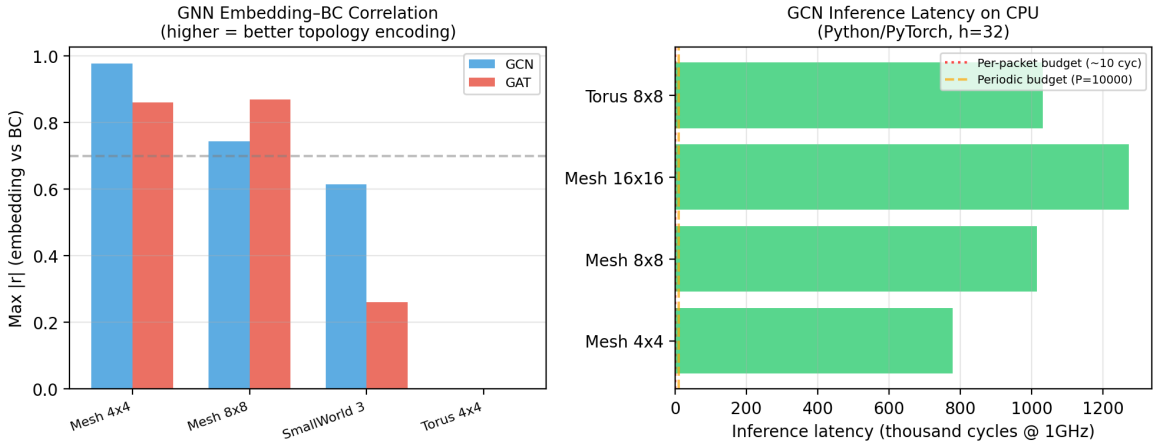

Fig. 4: Trái: Tương quan GNN embedding–BC. Phải: Inference latency đo được trên CPU (Python/PyTorch).

TABLE V: GNN inference latency on CPU (Python/PyTorch)

Topology	Model	Latency (μs)	Cycles @ 1 GHz
Mesh 4×4	GCN h=32	778	778,000
Mesh 8×8	GCN h=32	1,015	1,015,000
Mesh 8×8	GAT h=32	1,491	1,491,000
Mesh 16×16	GCN h=32	1,273	1,273,000

định kỳ: với $P = 10^5$ cycles và inference optimized (10^4–5×10^4 cycles), overhead là 10–50%.

D. Scalability Analysis

Đo inference time GCN từ 4×4 (16 nodes) đến 16×16 (256 nodes).

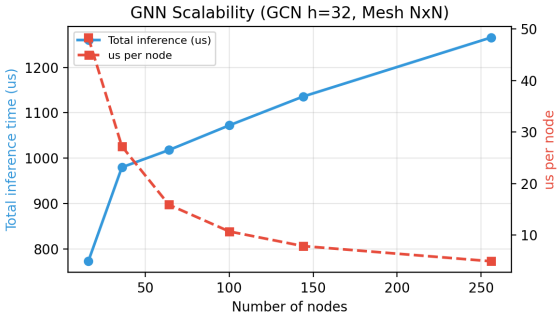

Fig. 5: GCN inference time scale O(N) với số nodes.

Inference time tăng từ 773 μs (16 nodes) lên 1,266 μs (256 nodes). Chi phí per-node giảm từ 48.3 xuống 4.9 μs/node.

Nhận xét: O(N) scaling dự đoán được, không bùng nổ.

VI. Discussion

A. Implications for Periodic Optimization

Inference latency (10^5–10^6 cycles) khẳng định per-packet GNN là bất khả thi. Tuy nhiên, tương quan GNN-BC cao ($|r| = 0.978$) cho thấy GNN representations mang thông tin topology hữu ích. Với periodic update $P = 10^5$ cycles và optimized inference (10^4–5×10^4 cycles), overhead là 10–50%.

B. The Generalization Challenge

GNN representations tổng quát hóa một phần (Pearson r cao) nhưng scale tuyệt đối phụ thuộc topology. Hai hướng: (1) topology-aware normalization, (2) fine-tuning strategies.

C. Limitations

1) **Static graph analysis:** Chưa có dynamic NoC state.
2) **Chưa tích hợp routing simulation:** Cần cycle-accurate simulator để đánh giá end-to-end.
3) **CPU-only measurements:** Trên server ARM, không phải embedded processor.
4) **Chưa có RTL validation.**
5) **Topology scope:** Chỉ 2D regular topologies.

D. Threats to Validity

- Tương quan không đồng nghĩa với quan hệ nhân quả.
- Python overhead làm tăng latency đo được.
- Synthetic features còn đơn giản, thiếu tính thực tế.

VII. Conclusion and Future Work

Bài báo trình bày nghiên cứu khả thi về tối ưu định tuyến định kỳ cho NoC sử dụng Graph Neural Networks. Ba kết quả thực nghiệm sơ bộ:

1) GNN embeddings tương quan mạnh với BC ($|r|$ lên đến 0.978).
2) Tổng quát hóa giữa các topology thành công một phần (Pearson $r \approx 0.96$).
3) Inference latency ($\approx 10^6$ cycles) khẳng định periodic optimization là cần thiết.

Hướng phát triển: (1) tích hợp GNN routing vào cycle-accurate simulator; (2) phát triển topology-aware normalization; (3) đánh giá quantized models trên embedded processors; (4) mở rộng sang irregular topologies.

References

[1] J. Hu and R. Marculescu, "DyAD – smart routing for networks-on-chip," *Proc. DAC*, 2004.
[2] Y. Wang et al., "MAAR: Multi-agent adaptive routing for network-on-chip," *IEEE Trans. Computers*, vol. 71, no. 8, pp. 1878–1891, 2022.
[3] R. R. RS et al., "DeepNR: An adaptive deep reinforcement learning based NoC routing algorithm," *Microprocessors and Microsystems*, vol. 92, 104499, 2022.
[4] H. Wei, Y. Zhao, and K. Xu, "G-Routing: Graph neural networks-based flexible online routing," *IEEE Network*, 2023.
[5] P. Almasan et al., "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *arXiv:1910.07421*, 2019.
[6] G. S. Malik and N. Kapre, "GraphNoC: Graph neural networks for application-specific FPGA NoC performance prediction," *Proc. FPT*, IEEE, 2024.
[7] V. Iyengar, V. H. Bui, and S. Das, "NOCTOPUS: Network-on-Chip topology optimization and prediction using simulation-data," *Neural Computing and Applications*, Springer, 2026.
[8] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021.
[9] M. Ebrahimi, M. Daneshtalab, and J. Plosila, "Regional-based routing for network-on-chip," *IEEE TCAD*, 2019.
[10] H. Zhang et al., "GARNN: Graph attention routing for network-on-chip," *Proc. DATE*, 2023.
[11] M. Besta et al., "Slim NoC: A low-diameter on-chip network topology for high energy efficiency and scalability," *Proc. ASPLOS*, ACM, 2018.
[12] P. Iff et al., "Sparse Hamming Graph: A customizable network-on-chip topology," *Proc. DAC*, 2023.
[13] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" *Proc. ICLR*, 2022.
[14] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.
[15] J. Duato, "A necessary and sufficient condition for deadlock-free routing in cut-through and store-and-forward networks," *IEEE Trans. Parallel and Distributed Systems*, vol. 7, no. 8, pp. 840–854, 1996.
[16] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," *ICLR Workshop*, 2019.
[17] A. Hagberg, P. Swart, and D. S. Chult, "Exploring network structure, dynamics, and function using NetworkX," *Proc. SciPy*, 2008.